# Sprint -8,9,10

## SPRINTS 8-11 — REMAINING FEATURES + POLISH

## PREREQUISITE: Sprints 1-7 complete

## PASTE ONE SPRINT AT A TIME INTO REPLIT AI AGENT

---

## SPRINT 8 — GOALS + CALENDAR OPTIMIZER + TIME TRACKING

## Features 15, 16, and 21

---

### FEATURE 15: AI GOAL & OKR SYSTEM

**tRPC Router:** `src/server/routers/goals.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const goalsRouter = router({
  list: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid(), level:
z.string().optional() }))
```

```
    .query(async ({ input }) => {
      return prisma.goal.findMany({
        where: { workspaceId: input.workspaceId, deletedAt: n
ull, ...(input.level ? { level: input.level } : {}) },
        include: { keyResults: true },
        orderBy: { createdAt: "desc" },
      });
    }),

  create: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      title: z.string().min(1),
      description: z.string().optional(),
      level: z.enum(["company", "team", "individual"]),
      parentGoalId: z.string().uuid().optional(),
      startDate: z.string(),
      targetDate: z.string(),
      linkedDatabaseIds: z.array(z.string().uuid()).optional
(),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.goal.create({
        data: { ...input, startDate: new Date(input.startDat
e), targetDate: new Date(input.targetDate), ownerId: ctx.use
r.id },
      });
    }),

  addKeyResult: protectedProcedure
    .input(z.object({
      goalId: z.string().uuid(),
      title: z.string(),
      metricType: z.enum(["number", "percentage", "currency",
"boolean"]),
      targetValue: z.number(),
```

```
      startValue: z.number().default(0),
      unit: z.string().optional(),
      autoTrackSource: z.string().optional(),
      autoTrackConfig: z.any().optional(),
    }))
    .mutation(async ({ input }) => {
      return prisma.keyResult.create({ data: input });
    }),

  updateKeyResultProgress: protectedProcedure
    .input(z.object({ keyResultId: z.string().uuid(), current
Value: z.number() }))
    .mutation(async ({ input }) => {
      const kr = await prisma.keyResult.findUnique({ where: {
id: input.keyResultId } });
      if (!kr) throw new Error("Key result not found");

      const progress = kr.targetValue !== kr.startValue
        ? Math.min(100, Math.max(0, ((input.currentValue - k
r.startValue) / (kr.targetValue - kr.startValue)) * 100))
        : 0;

      await prisma.keyResult.update({
        where: { id: input.keyResultId },
        data: { currentValue: input.currentValue, progressPer
cent: progress },
      });

      // Recalculate goal progress (average of all key result
s)
      const allKRs = await prisma.keyResult.findMany({ where:
{ goalId: kr.goalId } });
      const avgProgress = allKRs.reduce((sum, k) => sum + k.p
rogressPercent, 0) / allKRs.length;

      const goalStatus = avgProgress >= 100 ? "completed" : a
```

```
vgProgress < 30 ? "behind" : avgProgress < 60 ? "at_risk" :
"on_track";

      await prisma.goal.update({
        where: { id: kr.goalId },
        data: { progressPercent: avgProgress, status: goalSta
tus },
      });

      return { progress, goalStatus };
    }),

  // AI suggest key results for a goal
  suggestKeyResults: protectedProcedure
    .input(z.object({ goalTitle: z.string(), goalDescription:
z.string().optional() }))
    .mutation(async ({ input }) => {
      const result = await aiService.generateJSON<{ keyResult
s: Array<{ title: string; metricType: string; targetValue: nu
mber; unit: string }> }>(
        `Suggest 3-5 measurable key results for this goal:
"${input.goalTitle}" ${input.goalDescription ? `- ${input.goa
lDescription}` : ""}

Return: { keyResults: [{title, metricType: 'number'|'percenta
ge'|'currency'|'boolean', targetValue, unit}] }
Each key result must be specific and measurable.`,
        { systemPrompt: "Suggest measurable OKR key result
s.", temperature: 0.5 },
      );
      return result;
    }),

  // AI risk assessment for a goal
  assessRisk: protectedProcedure
    .input(z.object({ goalId: z.string().uuid() }))
```

```
    .mutation(async ({ input }) => {
      const goal = await prisma.goal.findUnique({
        where: { id: input.goalId },
        include: { keyResults: true },
      });
      if (!goal) throw new Error("Goal not found");

      const daysLeft = Math.ceil((goal.targetDate.getTime() -
Date.now()) / 86400000);

      const result = await aiService.generateText(
        `Assess risk for this goal:
"${goal.title}" — ${goal.progressPercent}% complete, ${daysLe
ft} days remaining
Key Results: ${JSON.stringify(goal.keyResults.map(kr => ({ ti
tle: kr.title, progress: kr.progressPercent })))}

Provide: risk level (low/medium/high), reasoning, and 2-3 spe
cific recommendations.`,
        { systemPrompt: "Assess goal risks and provide action
able recommendations.", maxTokens: 500, temperature: 0.3 },
      );

      await prisma.goal.update({ where: { id: input.goalId },
data: { aiRiskAssessment: result.text } });
      return { assessment: result.text };
    }),
});
```

## Goal UI: `src/app/(dashboard)/[workspaceId]/goals/page.tsx`

Build a page with:

- Goal hierarchy tree view (Company > Team > Individual)
- Each goal card: title, progress ring (0-100%), status badge, key results list

- Click goal → expand key results with progress bars
- "Add Goal" button → modal with form (title, level, parent, dates)
- "Add Key Result" → inline form within goal card
- "Suggest Key Results" button → AI generates suggestions
- "Assess Risk" button → AI risk assessment card

---

## FEATURE 16: AI CALENDAR OPTIMIZER

### tRPC Router: `src/server/routers/calendar-optimizer.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const calendarOptimizerRouter = router({
  // Find best meeting time for multiple participants
  findMeetingTime: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      participantIds: z.array(z.string().uuid()),
      durationMinutes: z.number().min(15).max(480),
      dateRange: z.object({ start: z.string(), end: z.string
() }),
      preferMorning: z.boolean().default(false),
    }))
    .mutation(async ({ input }) => {
      // Get all participants' busy times from calendar
      const busySlots = await Promise.all(
        input.participantIds.map(async (userId) => {
          const events = await prisma.meeting.findMany({
            where: {
              participants: { some: { userId } },
```

```
              startTime: { gte: new Date(input.dateRange.star
t), lte: new Date(input.dateRange.end) },
            },
            select: { startTime: true, endTime: true },
          });
          // Also get focus blocks
          const focusBlocks = await prisma.focusTimeBlock.fin
dMany({
            where: {
              userId,
              date: { gte: new Date(input.dateRange.start), l
te: new Date(input.dateRange.end) },
            },
          });
          return { userId, events, focusBlocks };
        })
      );

      // Get preferences
      const prefs = await Promise.all(
        input.participantIds.map(async (userId) => {
          return prisma.schedulingPreference.findUnique({ whe
re: { userId } });
        })
      );

      // Use AI to find optimal slots
      const result = await aiService.generateJSON<{ suggestio
ns: Array<{ date: string; startTime: string; endTime: string;
score: number; reason: string }> }>(
        `Find the best ${input.durationMinutes}-minute meetin
g slots for ${input.participantIds.length} people.

BUSY TIMES: ${JSON.stringify(busySlots.map(s => ({ userId: s.
userId, events: s.events.map(e => ({ start: e.startTime, end:
e.endTime })) })))}
```

```
PREFERENCES: ${JSON.stringify(prefs.map(p => ({ workingHours:
p?.workingHours, preferMeetingsIn: p?.preferMeetingsIn })))}
DATE RANGE: ${input.dateRange.start} to ${input.dateRange.en
d}
PREFER MORNING: ${input.preferMorning}

Return top 5 suggestions: { suggestions: [{date, startTime, e
ndTime, score: 0-100, reason}] }
Score based on: respects working hours, avoids focus blocks,
minimal disruption, consistent timing.`,
        { systemPrompt: "Find optimal meeting times.", temper
ature: 0.2 },
      );
      return result;
    }),

  // Get/set optimization rules
  getRules: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.calendarOptimizationRule.findMany({
        where: { workspaceId: input.workspaceId, isActive: tr
ue },
      });
    }),

  createRule: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      ruleType: z.enum(["no_meeting_window", "buffer_time",
"meeting_defrag", "focus_protection"]),
      config: z.any(),
      scope: z.enum(["workspace", "team", "individual"]).defa
ult("workspace"),
      scopeId: z.string().uuid().optional(),
    }))
```

```
    .mutation(async ({ input, ctx }) => {
      return prisma.calendarOptimizationRule.create({
        data: { ...input, createdBy: ctx.user.id },
      });
    }),
});
```

---

## FEATURE 21: AI TIME TRACKING

### tRPC Router: `src/server/routers/time-tracking.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";

export const timeTrackingRouter = router({
  // Log a manual time entry
  logTime: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      taskId: z.string().uuid().optional(),
      description: z.string().optional(),
      durationMinutes: z.number().min(1),
      startedAt: z.string(),
      endedAt: z.string().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.timeEntry.create({
        data: {
          ...input,
          userId: ctx.user.id,
          startedAt: new Date(input.startedAt),
          endedAt: input.endedAt ? new Date(input.endedAt) :
undefined,
```

```
          source: "manual",
        },
      });
    }),

  // Get time entries for a date range
  getEntries: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      startDate: z.string(),
      endDate: z.string(),
      taskId: z.string().uuid().optional(),
    }))
    .query(async ({ input, ctx }) => {
      return prisma.timeEntry.findMany({
        where: {
          userId: ctx.user.id,
          startedAt: { gte: new Date(input.startDate), lte: n
ew Date(input.endDate) },
          ...(input.taskId ? { taskId: input.taskId } : {}),
        },
        orderBy: { startedAt: "desc" },
      });
    }),

  // Get time analytics (grouped by day, project, etc.)
  getAnalytics: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      startDate: z.string(),
      endDate: z.string(),
    }))
    .query(async ({ input, ctx }) => {
      const entries = await prisma.timeEntry.findMany({
        where: {
          userId: ctx.user.id,
```

```
          startedAt: { gte: new Date(input.startDate), lte: n
ew Date(input.endDate) },
        },
        include: { task: { select: { title: true, databaseId:
true } } },
      });

      // Group by day
      const byDay: Record<string, number> = {};
      for (const entry of entries) {
        const day = entry.startedAt.toISOString().split("T")
[0];
        byDay[day] = (byDay[day] || 0) + entry.durationMinute
s;
      }

      // Group by task
      const byTask: Record<string, { title: string; minutes:
number }> = {};
      for (const entry of entries) {
        const key = entry.taskId || "no-task";
        if (!byTask[key]) byTask[key] = { title: entry.task?.
title || entry.description || "Untracked", minutes: 0 };
        byTask[key].minutes += entry.durationMinutes;
      }

      const totalMinutes = entries.reduce((sum, e) => sum +
e.durationMinutes, 0);

      return { totalMinutes, byDay, byTask: Object.values(byT
ask).sort((a, b) => b.minutes - a.minutes), entryCount: entri
es.length };
    }),

  // Get estimation accuracy (predicted vs actual)
  getEstimationAccuracy: protectedProcedure
```

```
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      const estimates = await prisma.taskDurationEstimate.fin
dMany({
        where: { workspaceId: input.workspaceId, actualMinute
s: { not: null } },
        take: 50,
        orderBy: { createdAt: "desc" },
      });

      const accuracy = estimates.map(e => ({
        estimated: e.estimatedMinutes,
        actual: e.actualMinutes!,
        error: Math.abs(e.estimatedMinutes - e.actualMinute
s!) / e.actualMinutes! * 100,
      }));

      const avgError = accuracy.length > 0 ? accuracy.reduce
((sum, a) => sum + a.error, 0) / accuracy.length : 0;

      return { estimates: accuracy, averageErrorPercent: Mat
h.round(avgError), totalEstimates: accuracy.length };
    }),
});
```

## VERIFICATION — SPRINT 8

```
□ Create company goal → appears in goal hierarchy
□ Add 3 key results → progress bars shown
□ Update key result value → goal progress auto-recalculates
□ "Suggest Key Results" → AI generates relevant suggestions
□ "Assess Risk" → AI risk analysis with recommendations
□ Find meeting time → top 5 suggestions with scores
□ Create "No Meeting Wednesday 10-12" rule → saved
□ Log time manually → entry appears in time tracking
```

```
□ View time analytics → daily breakdown chart, by-task breakd
own
□ Estimation accuracy view shows predicted vs actual comparis
on
```

---

# SPRINT 9 — NOTIFICATIONS + TEMPLATES + VOICE

# Features 20, 17, and 18

---

## FEATURE 20: AI SMART NOTIFICATIONS

### tRPC Router: `src/server/routers/notifications.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const notificationsRouter = router({
  // Get notifications (filtered by priority)
  getNotifications: protectedProcedure
    .input(z.object({
      priority: z.enum(["urgent", "important", "normal", "lo
w", "all"]).default("all"),
      unreadOnly: z.boolean().default(false),
      limit: z.number().default(50),
    }))
    .query(async ({ input, ctx }) => {
      return prisma.notificationItem.findMany({
        where: {
          userId: ctx.user.id,
```

```
          ...(input.priority !== "all" ? { aiPriority: input.
priority } : {}),
          ...(input.unreadOnly ? { isRead: false } : {}),
        },
        orderBy: [{ aiPriority: "asc" }, { createdAt: "desc"
}],
        take: input.limit,
      });
    }),

  // Mark as read
  markRead: protectedProcedure
    .input(z.object({ ids: z.array(z.string().uuid()) }))
    .mutation(async ({ input }) => {
      await prisma.notificationItem.updateMany({
        where: { id: { in: input.ids } },
        data: { isRead: true, readAt: new Date() },
      });
    }),

  // Get/update notification preferences
  getPreferences: protectedProcedure
    .query(async ({ ctx }) => {
      return prisma.notificationPreference.upsert({
        where: { userId: ctx.user.id },
        create: { userId: ctx.user.id, workspaceId: "" }, //
will be set on first real save
        update: {},
      });
    }),

  updatePreferences: protectedProcedure
    .input(z.object({
      alwaysNotify: z.array(z.string()).optional(),
      batchNotify: z.array(z.string()).optional(),
      muteNotify: z.array(z.string()).optional(),
```

```
      batchFrequency: z.string().optional(),
      batchTimes: z.array(z.string()).optional(),
      focusModeAction: z.string().optional(),
      aiTriageEnabled: z.boolean().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.notificationPreference.update({
        where: { userId: ctx.user.id },
        data: input,
      });
    }),

  // Get unread count by priority
  getUnreadCounts: protectedProcedure
    .query(async ({ ctx }) => {
      const counts = await prisma.notificationItem.groupBy({
        by: ["aiPriority"],
        where: { userId: ctx.user.id, isRead: false },
        _count: true,
      });
      return Object.fromEntries(counts.map(c => [c.aiPriorit
y, c._count]));
    }),
});
```

---

# FEATURE 17: AI TEMPLATE ENGINE

## tRPC Router: `src/server/routers/templates.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
```

```
export const templatesRouter = router({
  list: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid().optional(),
      category: z.string().optional(),
      isPublic: z.boolean().optional(),
    }))
    .query(async ({ input }) => {
      return prisma.template.findMany({
        where: {
          deletedAt: null,
          ...(input.category ? { category: input.category } :
{}),
          OR: [
            { isPublic: true },
            ...(input.workspaceId ? [{ workspaceId: input.wor
kspaceId }] : []),
          ],
        },
        orderBy: { usageCount: "desc" },
      });
    }),

  // AI generate template from description
  generateFromDescription: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      description: z.string(),
      type: z.enum(["page", "database", "workspace"]).default
("database"),
    }))
    .mutation(async ({ input, ctx }) => {
      const result = await aiService.generateJSON<{
        name: string; description: string; category: string;
        content: any; // structure depends on type
      }>(
```

```
        `Create a ${input.type} template for: "${input.descri
ption}"

${input.type === "database" ? `Return: { name, description, c
ategory, content: { properties: [{name, type, options?}], vie
ws: [{name, type, groupBy?}], sampleRows: [{}] } }` : ""}
${input.type === "page" ? `Return: { name, description, categ
ory, content: { blocks: [{type: 'heading'|'paragraph'|'lis
t'|'callout', content: string}] } }` : ""}

Categories: project_management, personal, marketing, engineer
ing, hr, sales, design, custom`,
        { systemPrompt: "Create practical, complete template
s.", temperature: 0.5 },
      );

      return prisma.template.create({
        data: {
          workspaceId: input.workspaceId,
          name: result.name,
          description: result.description,
          category: result.category,
          type: input.type,
          content: result.content,
          createdBy: ctx.user.id,
        },
      });
    }),

  // Use a template (increment usage count)
  useTemplate: protectedProcedure
    .input(z.object({ templateId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      return prisma.template.update({
        where: { id: input.templateId },
        data: { usageCount: { increment: 1 } },
```

```
      });
    }),

  // AI recommend templates based on user role/workspace
  getRecommendations: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.template.findMany({
        where: { isPublic: true },
        orderBy: { usageCount: "desc" },
        take: 10,
      });
    }),
});
```

---

## FEATURE 18: AI VOICE COMMANDS & NOTES

### tRPC Router: `src/server/routers/voice.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import OpenAI from "openai";

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KE
Y });

export const voiceRouter = router({
  // Transcribe audio
  transcribe: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      audioUrl: z.string(),
```

```
    }))
    .mutation(async ({ input, ctx }) => {
      const note = await prisma.voiceNote.create({
        data: {
          userId: ctx.user.id,
          workspaceId: input.workspaceId,
          audioUrl: input.audioUrl,
          durationSecs: 0,
          status: "transcribing",
        },
      });

      try {
        const audioResponse = await fetch(input.audioUrl);
        const audioBuffer = await audioResponse.arrayBuffer
();
        const audioFile = new File([audioBuffer], "voice.web
m", { type: "audio/webm" });

        const transcription = await openai.audio.transcriptio
ns.create({
          model: "whisper-1",
          file: audioFile,
        });

        const summary = await aiService.generateText(
          `Summarize this voice note in 1-2 sentences:\n"${tr
anscription.text}"`,
          { systemPrompt: "Summarize voice notes concisely.",
maxTokens: 100, temperature: 0.3 },
        );

        await prisma.voiceNote.update({
          where: { id: note.id },
          data: { transcript: transcription.text, aiSummary:
summary.text, status: "completed" },
```

```
        });

        return { id: note.id, transcript: transcription.text,
summary: summary.text };
      } catch (error: any) {
        await prisma.voiceNote.update({
          where: { id: note.id },
          data: { status: "failed" },
        });
        throw error;
      }
    }),

  // Convert voice note to task
  convertToTask: protectedProcedure
    .input(z.object({ voiceNoteId: z.string().uuid(), workspa
ceId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const note = await prisma.voiceNote.findUnique({ where:
{ id: input.voiceNoteId } });
      if (!note || !note.transcript) throw new Error("Voice n
ote not found or not transcribed");

      const parsed = await aiService.generateJSON<{ title: st
ring; description?: string; dueDate?: string; priority?: stri
ng }>(
        `Parse this voice note into a task:\n"${note.transcri
pt}"\nReturn: { title, description?, dueDate?, priority? }`,
        { systemPrompt: "Parse voice into structured task.",
temperature: 0.2 },
      );

      const task = await prisma.task.create({
        data: { ...parsed, workspaceId: input.workspaceId, cr
eatedBy: ctx.user.id, dueDate: parsed.dueDate ? new Date(pars
ed.dueDate) : undefined },
```

```
      });

      await prisma.voiceNote.update({
        where: { id: input.voiceNoteId },
        data: { convertedTo: "task", convertedId: task.id },
      });

      return task;
    }),

  // List voice notes
  list: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid(), limit:
z.number().default(20) }))
    .query(async ({ input, ctx }) => {
      return prisma.voiceNote.findMany({
        where: { userId: ctx.user.id, workspaceId: input.work
spaceId },
        orderBy: { createdAt: "desc" },
        take: input.limit,
      });
    }),
});
```

## VERIFICATION — SPRINT 9

```
□ Notification inbox shows items grouped by priority (urgent/
important/normal/low)
□ Mark notifications as read → disappear from unread view
□ Notification preferences saved and respected
□ Template gallery displays templates sorted by usage
□ "Generate from description" → AI creates full template → pr
eview shown
□ Apply template → creates page/database with template conten
t
```

```
□ Record voice note → transcription appears → summary shown
□ Convert voice note to task → task created with parsed field
s
□ Voice notes list shows all recordings with transcripts
```

---

# SPRINT 10 — DASHBOARD + TEAM INSIGHTS + INTEGRATIONS + ONBOARDING

# Features 23, 22, 24, and 25

---

## FEATURE 23: AI PERSONAL COMMAND CENTER

### tRPC Router: `src/server/routers/command-center.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const commandCenterRouter = router({
  // Get all dashboard data in one call (minimize API calls)
  getDashboard: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      const today = new Date();
      const todayStr = today.toISOString().split("T")[0];

      const [dailyPlan, habits, goals, notifications, meeting
s, recentActivity, focusSession, productivityScore] = await P
romise.all([
        // Today's plan
```

```
        prisma.dailyPlan.findUnique({ where: { userId_date: {
userId: ctx.user.id, date: new Date(todayStr) } } }),
        // Active habits with today's completion status
        prisma.habit.findMany({
          where: { userId: ctx.user.id, isActive: true, delet
edAt: null },
          include: { completions: { where: { date: new Date(t
odayStr) } } },
        }),
        // Active goals
        prisma.goal.findMany({
          where: { workspaceId: input.workspaceId, ownerId: c
tx.user.id, deletedAt: null, status: { not: "completed" } },
          include: { keyResults: true },
          take: 5,
        }),
        // Urgent + important notifications
        prisma.notificationItem.findMany({
          where: { userId: ctx.user.id, isRead: false, aiPrio
rity: { in: ["urgent", "important"] } },
          take: 10,
          orderBy: { createdAt: "desc" },
        }),
        // Today's meetings
        prisma.meeting.findMany({
          where: {
            participants: { some: { userId: ctx.user.id } },
            startTime: { gte: new Date(`${todayStr}T00:00:00
`), lt: new Date(`${todayStr}T23:59:59`) },
          },
          orderBy: { startTime: "asc" },
        }),
        // Recent activity in workspace
        prisma.task.findMany({
          where: { workspaceId: input.workspaceId, updatedAt:
{ gte: new Date(Date.now() - 86400000) } },
```

```
          include: { assignee: { select: { name: true } } },
          take: 10,
          orderBy: { updatedAt: "desc" },
        }),
        // Active focus session
        prisma.focusSession.findFirst({ where: { userId: ctx.
user.id, status: { in: ["active", "paused"] } } }),
        // Today's productivity score
        prisma.productivityScore.findUnique({ where: { userId
_date: { userId: ctx.user.id, date: new Date(todayStr) } }
}),
      ]);

      const planBlocks = (dailyPlan?.timeBlocks as any[]) ||
[];
      const completedBlocks = planBlocks.filter((b: any) =>
b.isCompleted).length;
      const totalBlocks = planBlocks.filter((b: any) => ["br
eak", "buffer"].includes(b.type)).length;

      return {
        dailyPlan: dailyPlan ? {
          blocksCount: totalBlocks,
          completedCount: completedBlocks,
          nextBlock: planBlocks.find((b: any) => !b.isComplet
ed && !b.isSkipped),
          summary: dailyPlan.aiSummary,
        } : null,
        habits: habits.map(h => ({
          id: h.id, name: h.name, icon: h.icon, streak: h.str
eakCurrent,
          completedToday: h.completions.length > 0,
        })),
        goals: goals.map(g => ({
          id: g.id, title: g.title, progress: g.progressPerce
nt, status: g.status,
```

```
        })),
        notifications: { urgentCount: notifications.filter(n
=> n.aiPriority === "urgent").length, importantCount: notific
ations.filter(n => n.aiPriority === "important").length, item
s: notifications },
        meetings: meetings.map(m => ({ id: m.id, title: m.tit
le, startTime: m.startTime })),
        recentActivity: recentActivity.map(t => ({ id: t.id,
title: t.title, assignee: t.assignee?.name, updatedAt: t.upda
tedAt })),
        focusSession,
        productivityScore,
      };
    }),

  // Get AI suggestion for "what to do next"
  getAiSuggestion: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const todayStr = new Date().toISOString().split("T")
[0];
      const plan = await prisma.dailyPlan.findUnique({ where:
{ userId_date: { userId: ctx.user.id, date: new Date(todaySt
r) } } });
      const nextBlock = plan ? (plan.timeBlocks as any[]).fin
d((b: any) => !b.isCompleted && !b.isSkipped) : null;

      const overdueTasks = await prisma.task.findMany({
        where: { assigneeId: ctx.user.id, dueDate: { lt: new
Date() }, completedAt: null, deletedAt: null },
        take: 3,
      });

      const result = await aiService.generateText(
        `What should this user do next?
Next planned block: ${nextBlock ? `"${nextBlock.title}" at
```

```
${nextBlock.startTime}` : "No plan"}
Overdue tasks: ${overdueTasks.map(t => t.title).join(", ") ||
"None"}
Current time: ${new Date().toLocaleTimeString()}

Give ONE specific, actionable suggestion in 1-2 sentences.`,
        { systemPrompt: "Give specific next-action suggestion
s.", maxTokens: 150, temperature: 0.5 },
      );
      return { suggestion: result.text };
    }),
});
```

### Command Center UI: `src/app/(dashboard)/[workspaceId]/command-center/page.tsx`

Build the dashboard exactly as specified in the overview:

- Top: Greeting with date
- 2×2 grid: Today's Plan card, Priority Inbox card, Streaks & Habits card, Goals card
- AI Suggestion card (full width)
- Bottom row: Upcoming meetings, Recent activity

---

## FEATURE 22: AI TEAM COLLABORATION INSIGHTS

### tRPC Router: `src/server/routers/team-insights.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const teamInsightsRouter = router({
  // Get team workload dashboard
```

```
  getTeamWorkload: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input }) => {
      const members = await prisma.workspaceMember.findMany({
        where: { workspaceId: input.workspaceId },
        include: { user: { select: { id: true, name: true, em
ail: true } } },
      });

      const memberStats = await Promise.all(members.map(async
(m) => {
        const activeTasks = await prisma.task.count({ where:
{ assigneeId: m.userId, completedAt: null, deletedAt: null }
});
        const overdueTasks = await prisma.task.count({ where:
{ assigneeId: m.userId, dueDate: { lt: new Date() }, complete
dAt: null, deletedAt: null } });
        const todayScore = await prisma.productivityScore.fin
dUnique({ where: { userId_date: { userId: m.userId, date: new
Date(new Date().toISOString().split("T")[0]) } } });

        const workloadScore = activeTasks > 12 ? "red" : acti
veTasks > 8 ? "yellow" : "green";

        return {
          userId: m.userId, name: m.user.name,
          activeTasks, overdueTasks,
          focusMinutes: todayScore?.focusMinutes || 0,
          meetingMinutes: todayScore?.meetingMinutes || 0,
          workloadScore,
        };
      }));

      return { members: memberStats };
    }),
```

```
  // Get AI team insights
  getInsights: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const snapshot = await prisma.teamInsightSnapshot.findF
irst({
        where: { workspaceId: input.workspaceId },
        orderBy: { date: "desc" },
      });

      if (!snapshot) return { insights: "Not enough data yet.
Team insights will appear after a few days of usage." };

      const result = await aiService.generateText(
        `Analyze this team data and provide 3 actionable insi
ghts:
${JSON.stringify(snapshot.memberStats)}
${JSON.stringify(snapshot.teamMetrics)}`,
        { systemPrompt: "Provide actionable team productivity
insights. Be specific.", maxTokens: 500, temperature: 0.4 },
      );
      return { insights: result.text };
    }),
});
```

---

## FEATURE 24: AI INTEGRATION HUB

### tRPC Router: `src/server/routers/integrations.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";

export const integrationsRouter = router({
```

```
  list: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.externalIntegration.findMany({ where: { w
orkspaceId: input.workspaceId } });
    }),

  connect: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      type: z.enum(["google_drive", "github", "gitlab", "jir
a", "linear", "slack", "figma", "confluence"]),
      displayName: z.string(),
      accessToken: z.string(),
      refreshToken: z.string().optional(),
      scopes: z.array(z.string()),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.externalIntegration.create({
        data: {
          ...input,
          accessTokenEncrypted: input.accessToken, // TODO: e
ncrypt in production
          refreshTokenEncrypted: input.refreshToken,
          createdBy: ctx.user.id,
        },
      });
    }),

  disconnect: protectedProcedure
    .input(z.object({ id: z.string().uuid() }))
    .mutation(async ({ input }) => {
      return prisma.externalIntegration.update({
        where: { id: input.id },
        data: { syncEnabled: false, syncStatus: "disconnecte
d" },
```

```
      });
    }),

  // Trigger manual sync
  syncNow: protectedProcedure
    .input(z.object({ id: z.string().uuid() }))
    .mutation(async ({ input }) => {
      await prisma.externalIntegration.update({
        where: { id: input.id },
        data: { syncStatus: "syncing" },
      });
      // Enqueue sync job
      return { status: "syncing" };
    }),
});
```

---

## FEATURE 25: AI ONBOARDING ASSISTANT

### tRPC Router: `src/server/routers/onboarding.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const onboardingRouter = router({
  getProgress: protectedProcedure
    .query(async ({ ctx }) => {
      return prisma.onboardingProgress.upsert({
        where: { userId: ctx.user.id },
        create: { userId: ctx.user.id, workspaceId: "" },
        update: {},
      });
    }),
```

```
  completeStep: protectedProcedure
    .input(z.object({ step: z.string() }))
    .mutation(async ({ input, ctx }) => {
      const progress = await prisma.onboardingProgress.findUn
iique({ where: { userId: ctx.user.id } });
      if (!progress) throw new Error("Progress not found");

      const steps = [...progress.completedSteps, input.step];
      const TOTAL_STEPS = ["welcome", "profile", "first_tas
k", "first_page", "join_channel", "first_habit", "ai_search",
"daily_plan"];
      const isComplete = TOTAL_STEPS.every(s => steps.include
s(s));

      return prisma.onboardingProgress.update({
        where: { userId: ctx.user.id },
        data: { completedSteps: steps, isComplete },
      });
    }),

  dismiss: protectedProcedure
    .mutation(async ({ ctx }) => {
      return prisma.onboardingProgress.update({
        where: { userId: ctx.user.id },
        data: { dismissedAt: new Date() },
      });
    }),

  // AI answers "how do I..." questions
  askHelp: protectedProcedure
    .input(z.object({ question: z.string() }))
    .mutation(async ({ input }) => {
      const result = await aiService.generateText(
        `Answer this user question about the productivity pla
tform:
```

```
"${input.question}"

Provide a clear, step-by-step answer. If it involves a specif
ic feature, explain how to access it and use it.`,
        { systemPrompt: "You are a helpful onboarding assista
nt for a productivity platform. Give clear, step-by-step inst
ructions.", maxTokens: 500, temperature: 0.4 },
      );
      return { answer: result.text };
    }),

  // AI suggests unused features
  getFeatureSuggestions: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      // Check which features the user has used
      const hasUsedWriting = await prisma.aiWritingHistory.co
unt({ where: { userId: ctx.user.id } }) > 0;
      const hasUsedSearch = await prisma.knowledgeQuery.count
({ where: { userId: ctx.user.id } }) > 0;
      const hasUsedPlan = await prisma.dailyPlan.count({ wher
e: { userId: ctx.user.id } }) > 0;
      const hasUsedFocus = await prisma.focusSession.count({
where: { userId: ctx.user.id } }) > 0;
      const hasUsedHabits = await prisma.habit.count({ where:
{ userId: ctx.user.id } }) > 0;

      const suggestions: Array<{ feature: string; reason: str
ing }> = [];
      if (!hasUsedWriting) suggestions.push({ feature: "AI Wr
iting", reason: "Press Space on empty lines to generate conte
nt with AI" });
      if (!hasUsedSearch) suggestions.push({ feature: "AI Sea
rch", reason: "Press Cmd+K to search everything or ask AI que
stions" });
      if (!hasUsedPlan) suggestions.push({ feature: "Daily Pl
```

```
anner", reason: "AI can auto-schedule your day based on tasks
and meetings" });
      if (!hasUsedFocus) suggestions.push({ feature: "Focus S
essions", reason: "Start a focus timer to track time and bloc
k notifications" });
      if (!hasUsedHabits) suggestions.push({ feature: "Habit
s", reason: "Create recurring habits and track streaks" });

      return { suggestions };
    }),
});
```

## VERIFICATION — SPRINT 10

```
□ Command Center dashboard loads with all widgets populated
□ "What should I do next?" AI suggestion is specific and acti
onable
□ Team workload shows all members with workload indicators (g
reen/yellow/red)
□ Team insights provides AI analysis
□ Integration hub lists available connections
□ Connect a test integration → shows in list with sync status
□ Onboarding progress tracks completed steps
□ "How do I create a habit?" → AI provides step-by-step answe
r
□ Feature suggestions show unused features
```

---

# SPRINT 11 — INTEGRATION TESTING + POLISH

```
□ ROUTER MERGE: Ensure all 25 tRPC routers are merged in _ap
p.ts
```

```
□ CROSS-FEATURE: Daily plan uses habits (Feature 14) + calend
ar (Feature 16) + tasks (Feature 6)
□ CROSS-FEATURE: Agents (Feature 4) can search knowledge (Fea
ture 2) and create tasks (Feature 6)
□ CROSS-FEATURE: Meeting action items (Feature 7) create task
s in databases (Feature 6)
□ CROSS-FEATURE: Command Center (Feature 23) aggregates all f
eature data
□ CROSS-FEATURE: Focus sessions (Feature 9) auto-log time (Fe
ature 21)
□ CROSS-FEATURE: Habit completions (Feature 14) update goal p
rogress (Feature 15)
□ PERFORMANCE: All pages load < 2 seconds
□ PERFORMANCE: AI responses start streaming < 3 seconds
□ PERFORMANCE: Knowledge search returns results < 500ms
□ MOBILE: All pages responsive on 375px width
□ ERRORS: Every API has proper error handling (try/catch + us
er-friendly messages)
□ LOADING: Every async operation shows loading state
□ EMPTY: Every list/view shows appropriate empty state
□ KEYBOARD: Cmd+K (search), Cmd+J (AI), Cmd+Shift+D (daily pl
an), Cmd+N (new task)
□ TypeScript: npx tsc --noEmit → zero errors
□ README: Create/update README-AI-FEATURES.md documenting all
25 features
```

**PROJECT COMPLETE.**